

Aula 3 (Módulo 01) — Processos, systemd e cron na prática

1. `ps` de verdade — lendo a árvore de processos

Já usamos `ps -ef` na Aula 1 do Módulo 00. Agora vamos entender cada coluna e, principalmente, a **relação pai/filho**.

```
ps -ef
```

Colunas principais:

Coluna	Significado	Por que importa
UID	usuário dono do processo	processo de sistema rodando como usuário comum (ou vice-versa) é suspeito
PID	identificador único do processo	usado para referenciar o processo (ex.: <code>kill <PID></code>)
PPID	PID do processo pai (quem o iniciou)	a base da árvore de processos — quem criou o quê
CMD	comando completo, com argumentos	a "command line" — onde aparecem sinais de ofuscação, scripts, flags suspeitas

Para ver a árvore de processos de forma visual (pai → filho), use:

```
ps -ef --forest
```

Por que importa para segurança: a árvore de processos é uma das telas mais analisadas em qualquer investigação de endpoint. Um exemplo muito estudado no mundo Blue Team: `winword.exe` → `powershell.exe` (Word abrindo PowerShell como filho) no Windows, ou no Linux: um servidor web (`apache2` / `nginx`) que, de repente, tem como processo filho um `/bin/bash` ou `wget` — algo que um servidor web não faz em uso normal. Isso costuma indicar exploração de uma vulnerabilidade na aplicação (o atacante conseguiu executar comandos através dela).

2. `top` / `htop` — visão ao vivo

```
top
```

Mostra, atualizado a cada poucos segundos: uso de CPU, uso de memória, e os processos que mais consomem recursos, no topo. Aperte `q` para sair.

Por que importa para segurança: picos repentinos e constantes de CPU (ex.: um processo desconhecido usando 100% de CPU o tempo todo) é o padrão clássico de **cryptomining não autorizado** — um dos usos mais comuns de máquinas comprometidas hoje em dia (o atacante "aluga" o processamento da vítima para minerar criptomoeda).

3. `systemd` e `systemctl` — controlando serviços

Vimos na Aula 3 do Módulo 00 que serviço = processo que roda sempre em background. Na maioria das distribuições Linux modernas, quem gerencia isso é o **systemd**, e o comando para interagir com ele é `systemctl`.

```
systemctl list-units --type=service --state=running    # serviços ativos agora
systemctl status ssh                                  # status de um serviço específico
sudo systemctl start meuservico                        # iniciar
sudo systemctl stop meuservico                        # parar
sudo systemctl enable meuservico                      # ativar para iniciar automaticamente no boot
```

Por que importa para segurança: `systemctl enable` é literalmente o comando que garante que algo sobreviva a um reboot — exatamente a definição de **persistência** que mencionamos na Aula 3 do Módulo 00 (MITRE ATT&CK T1543). Um serviço novo e desconhecido, com `enable` ativo, apontando para um binário em `/tmp` ou `/home`, é um forte indicador de comprometimento. Os arquivos de definição de serviço ficam em `/etc/systemd/system/` — vale a pena, numa investigação, olhar o que existe ali e comparar com o que é esperado no ambiente.

4. `cron` — tarefas agendadas

`cron` é o "agendador de tarefas" do Linux: executa comandos automaticamente em horários/intervalos definidos, mesmo sem ninguém logado.

```
crontab -l      # lista as tarefas agendadas do usuário atual
```

Formato de uma linha de crontab:

```
* * * * * comando
| | | | |
| | | | └─ dia da semana (0-6, domingo=0)
| | | └─── mês (1-12)
| | └───── dia do mês (1-31)
| └────── hora (0-23)
└──────── minuto (0-59)
```

Exemplo: `0 3 * * * /home/user/backup.sh` roda o script `backup.sh` todo dia às 3h da manhã.

Por que importa para segurança: assim como `systemd`, `cron` é um mecanismo de **persistência** clássico — sobrevive a reboot e não depende de login. É catalogado no MITRE ATT&CK como **T1053.003 – Scheduled Task/Job: Cron**. Uma entrada de cron desconhecida, especialmente rodando com frequência alta ou apontando para um script em local incomum, é algo que todo analista Linux aprende a procurar. Além do `crontab` de cada usuário, existem também `/etc/cron.d/`, `/etc/crontab` e `/etc/cron.daily|weekly|monthly/` — vale a pena, numa investigação real, checar todos esses lugares, não só o `crontab -l` do usuário atual.

5. Conectando tudo

```
ps -ef --forest    → árvore de processos (quem criou quem)
top                → uso de recursos em tempo real (picos = suspeita)
systemctl         → controla o que roda sempre (persistência via serviço)
crontab -l        → controla o que roda em horário agendado (persistência via cron)
```

Esses quatro comandos, juntos, já cobrem boa parte da primeira hora de qualquer investigação em um host Linux suspeito.

6. Exercício prático

```
ps -ef --forest | head -20
systemctl list-units --type=service --state=running | head -10
crontab -l
```

Se `crontab -l` disser "no crontab for ", isso é normal — significa que você não tem nenhuma tarefa agendada. É exatamente esse tipo de saída "vazia" que você quer confirmar como linha de base normal, antes de comparar com um host que pode estar comprometido.

7. Recapitulando

- `ps -ef --forest` revela a árvore de processos (pai → filho) — a base de detecção de comportamento anômalo de processo.
- `top` mostra consumo de recursos ao vivo — picos incomuns podem indicar cryptomining ou outro processo malicioso.
- `systemctl` gerencia serviços — `enable` é o comando que cria persistência via serviço (T1543).
- `crontab` / `cron` agenda tarefas — mecanismo clássico de persistência (T1053.003).

Próxima aula

SSH — como conexões remotas funcionam, chaves vs. senha, e por que tentativas de login SSH são um dos eventos mais monitorados em qualquer ambiente exposto à internet.

Fontes recomendadas

- **freedesktop.org — documentação oficial do systemd**: referência completa de `systemctl` e unidades de serviço. <https://www.freedesktop.org/software/systemd/man/latest/systemctl.html>
- **MITRE ATT&CK — T1053.003 (Scheduled Task/Job: Cron)**: técnica oficial de persistência via cron. <https://attack.mitre.org/techniques/T1053/003/>
- **MITRE ATT&CK — T1543 (Create or Modify System Process)**: técnica oficial de persistência via serviço (já citada na Aula 3 do Módulo 00). <https://attack.mitre.org/techniques/T1543/>

